

Test-Time Adaptation via Sparse Delta Learning

Anonymous Authors¹

Abstract

Machine learning models deployed in dynamic, open-world environments often encounter distribution shifts that degrade performance, yet collecting labeled target data is costly or infeasible. Test-Time Adaptation (TTA) addresses this by updating models on unlabeled test data, but existing methods either adapt all parameters indiscriminately or rely on fixed update schemes, risking instability and inefficiency. We introduce **Sparse Delta Learning (SDL)**, a two-stage framework that performs *targeted, efficient adaptation*. In the first stage, *Dynamic Sparse Structure Discovery*, task-relevant parameter shifts are identified via differential inclusion dynamics with Inverse Scale Space properties, automatically filtering noise and enforcing sparsity. In the second stage, *Salient Structure Adaptation*, non-essential parameters are frozen, and the discovered subspace is fine-tuned to correct shrinkage bias, achieving functional optimality. SDL is compatible with arbitrary pre-trained models and differentiable adaptation objectives. Extensive experiments demonstrate that SDL serves as an effective plug-and-play framework, seamlessly boosting the performance of various TTA baselines while maintaining a superior trade-off between accuracy, stability, and parameter efficiency.

1. Introduction

Modern machine learning models are increasingly deployed in dynamic, open-world environments, where test data often diverges significantly from the training distribution. Such distribution shifts can severely degrade model performance, yet collecting labeled target data or retraining models is frequently impractical. To address this challenge, Test-Time Adaptation (TTA) has emerged as a critical paradigm. By

¹Anonymous Institution, Anonymous City, Anonymous Region, Anonymous Country. Correspondence to: Anonymous Author <anon.email@domain.com>.

Preliminary work. Under review by the International Conference on Machine Learning (ICML). Do not distribute.

adapting pre-trained models using solely unlabeled test data during deployment, TTA provides a practical mechanism for enhancing robustness in real-world systems.

The TTA landscape encompasses a variety of settings, primarily distinguished by the accessibility and temporal structure of the test data. Recent surveys (Liang et al., 2025) categorize these protocols based on data granularity: Test-Time Batch Adaptation (TTBA), which updates on isolated mini-batches; Online Test-Time Adaptation (OTTA), which continuously processes incoming data streams; and Test-Time Domain Adaptation (TTDA), which leverages the full unlabeled test dataset prior to inference. In this work, we focus on model-centric adaptation in online and domain adaptation scenarios, and do not consider batch-level adaptation strategies (e.g., TTBA). Since the inference in TTBA is influenced by other instances in the batch, it becomes highly unstable under the small batch sizes, we exclude it to avoid results that fluctuate based on the specific composition of the mini-batch for fair comparison.

A broad range of TTA strategies has been explored, including entropy minimization (Wang et al., 2020; Xia et al., 2022; Agarwal et al., 2022), pseudo-labeling (Liang et al., 2020; 2021; Iwasawa & Matsuo, 2021; Jang et al., 2022), consistency regularization (Sun et al., 2020; 2023; Ding et al., 2023), and memory-based updates (Cao et al., 2021; Chen et al., 2022). While these methods have demonstrated robust empirical performance across various benchmarks, they predominantly rely on static adaptation protocols, often assuming that updating either the full model or a fixed subset of parameters (e.g., normalization layers) suffices across diverse shifts. This one-size-fits-all approach overlooks the need to tailor parameter updates and connectivity patterns to the specific characteristics of different domains and tasks.

Recent studies have highlighted the dangers of unrestricted adaptation, particularly in online and continual settings, where unregulated updates can lead to instability and catastrophic forgetting. To mitigate these risks, various techniques have been proposed, including stochastic restoration (Wang et al., 2022b), Fisher-based regularization (Niu et al., 2022), and shift-aware parameter grouping (Choi et al., 2022). While these methods alleviate catastrophic forgetting and improve stability, they remain fundamentally agnostic to the underlying sparsity structure of the adaptation task,

relying on global penalties or randomized constraints that suppress parameter updates indiscriminately. As a result, models are often unable to fully exploit task-specific subspaces critical for optimal adaptation.

To overcome these limitations and strike a balance between performance and efficiency, we introduce a novel framework, dubbed **SDL**, which reformulates TTA as a **Sparse Delta Learning** problem. As shown in Figure 1, the adaptation proceeds in two coordinated stages. In the first stage, *Dynamic Sparse Structure Discovery*, we identify a sparse additive shift capturing only the essential modifications required for the target domain. This discovery process is modeled as a dynamic system governed by *Differential Inclusions* with the Inverse Scale Space (ISS) property (Burger et al., 2006; Osher et al., 2016). By leveraging these dynamics, *task-relevant features with persistent gradients emerge early, while stochastic noise is naturally suppressed*. Early stopping then filters redundant parameters, ensuring adherence to a predefined sparsity budget. Crucially, this formulation is flexible and compatible with arbitrary differentiable TTA loss functions. In the second stage, *Salient Structure Adaptation*, we transition from structure discovery to functional optimality. The sparsity-inducing dynamics in the first phase introduce shrinkage bias in the parameter magnitudes; to correct this, we freeze non-essential parameters and fine-tune only the discovered salient weights. This decoupling allows precise adaptation within the optimal sparse subspace, effectively eliminating bias and achieving task-specific fine-tuning.

Our two stages enable robust, efficient, and targeted adaptation, combining the stability benefits of constrained updates with the flexibility to exploit task-specific structures. We thus address key limitations of existing TTA methods, providing a principled approach to dynamic, sparse and effective test-time learning. Our contributions are summarized as below.

- We reformulate TTA as a sparse delta learning problem, which decouples adaptation into two coordinated stages: *Sparse Structure Discovery* and *Salient Structure Adaptation*. By combining the Inverse Scale Space dynamics to identify task-essential sparse structure with a subsequent masked refinement mechanism to rectify regularization-induced shrinkage, our approach ensures the model achieves functional optimality within a precisely located sparse subspace.
- Benefiting from its generalized gradient-based formulation, SDL acts as a **plug-and-play** framework. It can be seamlessly integrated with arbitrary pre-trained backbones and combined with various adaptation objectives to automatically identify the optimal adaptation subspace without modifying the original model architecture.

- Extensive experiments on widely-used adaptation benchmarks substantiate that our framework significantly enhances the performance of existing TTA baselines. Notably, on the Office benchmark, our method surpasses full-model fine-tuning while updating only **0.0022%** of total parameters, achieving a superior trade-off between adaptation efficiency and robust accuracy.

2. Related Work

Test Time Adaptation Test-Time Adaptation (TTA) enables pre-trained models to adjust to arbitrary target distributions during inference without accessing source data. A primary stream of research focuses on self-supervised optimization using the model’s own predictions. Prominently, Tent (Wang et al., 2020) updates normalization parameters by minimizing prediction entropy, while SHOT (Liang et al., 2020) exploits information maximization and pseudo-labeling to align target representations with the source hypothesis. Extending this direction, methods like AdaContrast (Chen et al., 2022) and MEMO (Zhang et al., 2022) further refine adaptation via augmentation consistency or nearest-neighbor mechanisms. Beyond prediction-based objectives, TTT (Sun et al., 2020) and its variants (e.g., TTT-MAE (Gandelsman et al., 2022)) utilize auxiliary tasks such as rotation prediction or masked reconstruction to update feature extractors. Finally, to address non-stationary data streams in online settings, approaches like CoTTA (Wang et al., 2022a) introduce mechanisms to mitigate catastrophic forgetting and stabilize continuous adaptation.

Linearized Bregman Iteration and Inverse Scale Space (ISS). The Linearized Bregman Iteration (LBI) (So et al., 2008; Osher et al., 2005) was proposed in image denoising. From the perspective of differential inclusion, (Osher et al., 2016) showed that LBI generates a solution path from sparse to dense. Besides, (Osher et al., 2005; 2016) discussed the *Inverse Scale Space* (ISS) property, meaning that important features are selected earlier in the solution path. In this paper, we leverage the ISS property to reformulate TTA as a continuous process of sparse structure discovery, filtering domain noise through temporal evolution.

3. Methodology

In this section, we first provide a formal definition of the Test-Time Adaptation (TTA) problem and review the distinct protocols in offline and online settings. We then formulate TTA as a sparsity-aware optimization problem and introduce our dynamic decoupling strategy based on Inverse Scale Space (ISS) dynamics. Finally, we present our two-stage framework Sparse Delta Learning (SDL), which sequentially discovers task-essential sparse structures and performs masked refinement to achieve efficient, accurate, and bias-free adaptation.

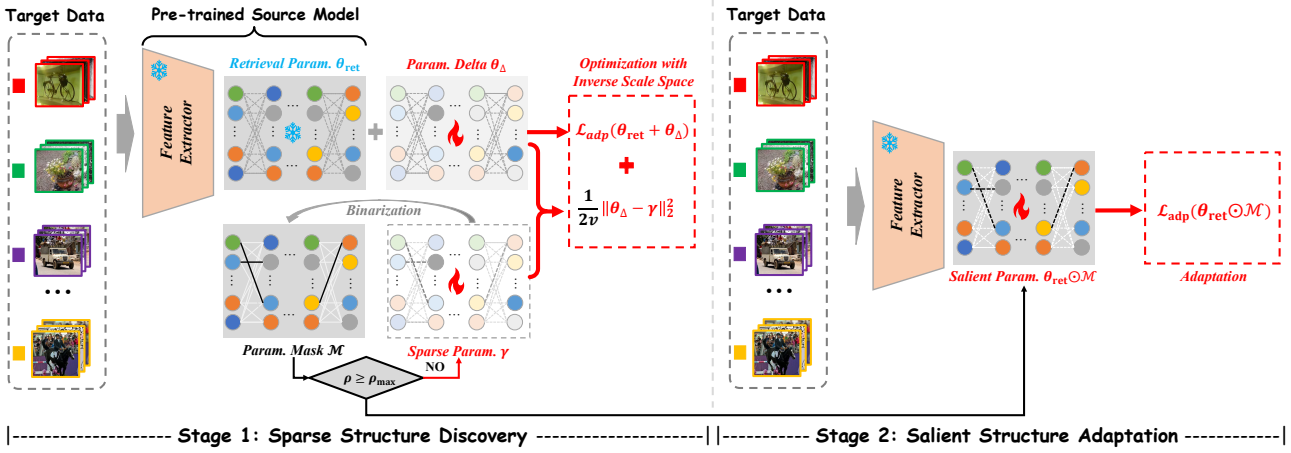


Figure 1. Overview of the proposed TTA framework with Sparse Delta Learning. Our framework consists of two synergistic stages. In Stage 1, we introduce a principled mechanism to dynamically identify a sparse adaptation subspace. This is achieved through differential inclusion dynamics characterized by the Inverse Scale Space property, allowing for the progressive discovery of salient parameters. To mitigate the shrinkage bias inherent in the structure discovery process, Stage 2 performs a decoupled fine-tuning phase that optimizes only the identified salient parameters, effectively separating sparsity induction from task-specific adaptation.

3.1. Test-Time Adaptation Problem Definition

We consider the standard test-time adaptation problem, where a labeled source dataset $\mathcal{D}_s$ and an unlabeled target dataset $\mathcal{D}_t$ are given, with an underlying distribution shift between them. Let $f_{\theta_{src}}$ denote a source model pre-trained on the source domain $\mathcal{D}_s$. The goal of TTA is to leverage the unlabeled target data $\mathcal{D}_t$, to achieve robust predictions under this shift (Liang et al., 2025), which can be formulated as:

$$\min_{\theta} \mathcal{L}_{adp}(\mathcal{D}_t; \theta), \quad \text{s.t.} \quad \theta = \theta_{src}. \quad (1)$$

We focus on two principal TTA protocols, distinguished by the availability and temporal structure of the target data:

Test-Time Domain Adaptation (Offline): In this setting, the complete unlabeled target dataset $\mathcal{D}_t = \{x_i\}_{i=1}^M$ is available prior to inference. Assuming a stationary target distribution, the model can be updated over multiple optimization epochs to produce a globally adapted model that maximizes performance across the entire target domain.

Online Test-Time Adaptation: In contrast, the online setting operates under a streaming paradigm, where target samples arrive sequentially in batches $\{\mathcal{B}_1, \mathcal{B}_2, \dots\}$. Each batch is observed only once, and the target distribution may drift over time. This setting requires models to adapt *on-the-fly*, emphasizing efficient, stable, and localized parameter updates to maintain robust predictions under non-stationary conditions.

3.2. Test Time Adaptation with Sparse Delta Learning

Standard test-time adaptation typically updates all model parameters, which is computationally expensive and prone to catastrophic forgetting. To improve efficiency while main-

taining adaptation performance, we reformulate TTA as a sparsity-aware adaptation problem.

Let θ_{src} denote the full parameter set of the pre-trained source model. We decouple θ_{src} into two disjoint subsets: the *frozen parameters* θ_{fix} , which remain unchanged to preserve the generic source knowledge, and the *retrieval parameters* θ_{ret} , which constitute the candidate subspace for fine-tuning. Instead of directly updating θ_{ret} , we learn a sparse additive delta θ_{Δ} on top of it. The adapted model is then constructed as:

$$\theta_{adp} = \theta_{fix} \cup (\theta_{ret} + \theta_{\Delta}).$$

To ensure parameter efficiency, we encourage the updates to remain sparse, controlling the fraction of active parameters within the retrieval set via a sparsity budget ρ_{max} :

$$\min_{\theta_{\Delta}} \mathcal{L}_{adp}(\mathcal{D}_t; \theta_{fix} \cup (\theta_{ret} + \theta_{\Delta})) \quad \text{s.t.} \quad \frac{\|\theta_{\Delta}\|_0}{\text{numel}(\theta_{ret})} \leq \rho_{max}, \quad (2)$$

where $\mathcal{L}_{adp}$ is the adaptation loss on the target data $\mathcal{D}_t$, and $\text{numel}(\cdot)$ counts the total number of parameters in the tensor. Unfortunately, directly optimizing the ℓ_0 -norm constraint is NP-hard for deep networks, motivating a dynamic approach to sparsity. We thus develop our contributions next.

3.3. Stage 1: Sparse Structure Discovery

To address this, we propose a **dynamic decoupling strategy** inspired by Inverse Scale Space (ISS) dynamics for sparse structure learning (Burger et al., 2006; Osher et al., 2016). Rather than treating sparse adaptation as a static optimization, we model it as a *bi-variable evolutionary process*. Specifically, we introduce a sparse proxy variable γ alongside the task-specific update θ_{Δ} , allowing us to decouple

Algorithm 1 Non-Restart Sparse Delta Learning for TTDA

```

1: Input: Target Dataset  $\mathcal{D}_t$ ; pretrained source parameters  $\theta_{\text{src}} = \theta_{\text{fix}} \cup \theta_{\text{ret}}$ ; step size  $\alpha$ ; damping factor  $\kappa > 0$ ; mask threshold  $\tau$ ; sparsity factor  $\nu$ ; sparsity budget  $\rho_{\text{max}}$ ; learning rate  $\eta$ ; momentum  $\omega \in (0, 1]$ .
2: Initialize parameters:  $\theta_{\Delta} \leftarrow \mathbf{0}, \gamma \leftarrow \mathbf{0}, z \leftarrow \mathbf{0}$ 
3: while optimization do
4:   for each batch  $\mathcal{B}_i$  in  $\mathcal{D}_t$  do
5:     Compute sparsity loss:
6:      $\mathcal{L}_{\text{adp}}(\mathcal{B}_i; \theta_{\text{fix}} \cup (\theta_{\text{ret}} + \theta_{\Delta})) + \frac{1}{2\nu} \|\theta_{\Delta} - \gamma\|_2^2$ 
7:     Perform SGD to update  $\theta_{\Delta}, z, \gamma$  as Eq.(5):
8:     if  $\frac{\|\gamma\|_0}{\text{numel}(\gamma)} \geq \rho_{\text{max}}$  then
9:       break
10:    end if
11:  end for
12:  Construct salient mask:  $\mathcal{M} = \mathbb{I}(|\gamma| \geq \tau)$ 
13:  Initialize retrieval parameters:  $\theta_{\text{ret}} \leftarrow \theta_{\text{ret}} + \theta_{\Delta}$ 
14:  while training do
15:    for each batch  $\mathcal{B}_i$  in  $\mathcal{D}_t$  do
16:      Compute adaptation loss:  $\mathcal{L}_{\text{adp}}(\mathcal{B}_i; \theta_{\text{fix}} \cup \theta_{\text{ret}})$ 
17:      Perform SGD with  $\mathcal{M}$  to update  $\theta_{\text{ret}}$ :
18:       $\theta_{\text{ret}} \leftarrow \theta_{\text{ret}} - \eta \cdot (\nabla_{\theta_{\text{ret}}} \mathcal{L}_{\text{adp}} \odot \mathcal{M})$ 
19:    end for
20:  end while
21: Return: Updated parameters:  $\theta_{\text{final}} = \theta_{\text{fix}} \cup \theta_{\text{ret}}$ 
    
```

Sparse Structure Discovery

Salient Structure Adaptation

Algorithm 2 Restart Sparse Delta Learning for OTTA

```

1: Input: Streaming batch  $\{\mathcal{B}_i\}$  from  $\mathcal{D}_t$ ; source parameters  $\theta_{\text{src}} = \theta_{\text{fix}} \cup \theta_{\text{ret}}$ ; step size  $\alpha$ ; damping factor  $\kappa > 0$ ; mask threshold  $\tau$ ; sparsity factor  $\nu$ ; sparsity budget  $\rho_{\text{max}}$ ; learning rate  $\eta$ ; momentum  $\omega \in (0, 1]$ .
2: Initialize parameters:  $\theta_{\Delta} \leftarrow \mathbf{0}, \gamma \leftarrow \mathbf{0}, z \leftarrow \mathbf{0}$ 
3: for each batch  $\mathcal{B}_i$  do
4:   Reset  $\theta_{\Delta} \leftarrow \mathbf{0}, \gamma \leftarrow \mathbf{0}, z \leftarrow \mathbf{0}$ 
5:   while optimization do
6:     Compute sparsity loss:
7:      $\mathcal{L}_{\text{adp}}(\mathcal{B}_i; \theta_{\text{fix}} \cup (\theta_{\text{ret}} + \theta_{\Delta})) + \frac{1}{2\nu} \|\theta_{\Delta} - \gamma\|_2^2$ 
8:     Perform SGD to update  $\theta_{\Delta}, z, \gamma$  as Eq.(5):
9:     if  $\frac{\|\gamma\|_0}{\text{numel}(\gamma)} \geq \rho_{\text{max}}$  then
10:      break
11:    end if
12:   end while
13:   Construct salient mask:  $\mathcal{M} = \mathbb{I}(|\gamma| \geq \tau)$ 
14:   Initialize retrieval parameters:  $\theta_{\text{ret}} \leftarrow \theta_{\text{ret}} + \theta_{\Delta}$ 
15:   Compute adaptation loss:  $\mathcal{L}_{\text{adp}}(\mathcal{B}_i; \theta_{\text{fix}} \cup \theta_{\text{ret}})$ 
16:   Update  $\theta_{\text{ret}}$  with  $\mathcal{M}$  and  $\omega$  through Moving Average:
17:    $\theta_{\text{ret}} \leftarrow \theta_{\text{ret}} + \omega(\theta_{\Delta} - \eta \cdot \nabla_{(\theta_{\text{ret}} + \theta_{\Delta})} \mathcal{L}_{\text{adp}} \odot \mathcal{M})$ 
18: Return: Updated parameters:  $\theta_{\text{final}}^i = \theta_{\text{src}} \cup \theta_{\text{ret}}$ 
19: end for
    
```

Sparse Structure Discovery

Salient Structure Adaptation

the objective into two roles: 1) Task Fitting: θ_{Δ} minimizes the task loss $\mathcal{L}_{\text{task}}$ to maintain predictive performance; 2) Structure Discovery: γ acts as a sparse structural proxy, identifying the most salient parameter directions. To link functional updates with the sparse structure, we introduce a *proximal coupling term*, yielding the following unconstrained objective:

$$\min_{\theta_{\Delta}, \gamma} \bar{\mathcal{L}}(\theta_{\Delta}, \gamma) = \mathcal{L}_{\text{adp}}(\mathcal{D}_t; \theta_{\text{fix}} \cup (\theta_{\text{ret}} + \theta_{\Delta})) + \frac{1}{2\nu} \|\theta_{\Delta} - \gamma\|_2^2, \quad (3)$$

where sparsity factor $\nu > 0$ controls the coupling strength between θ_{Δ} and its sparse proxy γ .

While the above objective is smooth, structural sparsity is enforced through dynamic evolution. We model the process as a differential inclusion:

$$\begin{aligned} \frac{d\theta_{\Delta}}{dt} &= -\kappa \nabla_{\theta_{\Delta}} \bar{\mathcal{L}}(\theta_{\Delta}, \gamma), \\ \frac{dz}{dt} &= -\nabla_{\gamma} \bar{\mathcal{L}}(\theta_{\Delta}, \gamma), \quad \text{s.t. } z(t) \in \partial J(\gamma(t)), \end{aligned} \quad (4)$$

where z denotes the dual variable; and κ is the damping factor with $J(\gamma) = \|\gamma\|_1$ promoting the sparsity.

This continuous formulation embodies the Inverse Scale Space (ISS) property: the dual variable z accumulates gradient signals arising from the coupling term. Gradients corresponding to consistent, task-relevant directions quickly build up in z , allowing the associated structural proxy γ to emerge early via the sub-gradient condition. In contrast, stochastic or noisy gradients tend to cancel out over

time, suppressing the activation of irrelevant parameters. In this way, the dynamics naturally perform *dynamic structure discovery*, effectively filtering noise and isolating the task-essential sparse subspace through temporal evolution.

Discretization. To implement this efficiently, we discretize the differential system using Euler steps with step size α starting from $\theta_{\Delta} = \gamma = z = \mathbf{0}$:

$$\begin{aligned} \theta_{\Delta}^{k+1} &= \theta_{\Delta}^k - \alpha \kappa \nabla_{\theta_{\Delta}} \bar{\mathcal{L}}(\theta_{\Delta}^k, \gamma^k), \\ z^{k+1} &= z^k - \alpha \nabla_{\gamma} \bar{\mathcal{L}}(\theta_{\Delta}^k, \gamma^k), \\ \gamma^{k+1} &= \kappa \cdot \text{prox}_J(z^{k+1}), \end{aligned} \quad (5)$$

where k denotes the iteration step, and $\text{prox}_{J(\cdot)}(z) := \arg \min_{\gamma} \frac{1}{2} \|\gamma - z\|_2^2 + J(\gamma) = \text{sign}(z) \otimes \max(|z| - 1, 0)$ is the proximal operator mapping the dual accumulator z back to the sparse structure γ .

Since the structural proxy γ is initialized as a null vector ($\gamma_0 = \mathbf{0}$), this iterative process gradually transfers gradient information from the dense parameter θ_{Δ} to γ , progressively activating the most significant weights. The evolution follows a natural “sparse-to-dense” trajectory: as $k \rightarrow \infty$, γ^k would eventually become fully dense with all elements being non-zero. However, our objective is to adapt the model within a limited parameter budget rather than to convergence. To enforce sparsity, we monitor the density of γ during the iterations and *terminate the process once the fraction of active parameters reaches the pre-defined sparsity budget* ρ_{max} :

$$\frac{\|\gamma\|_0}{\text{numel}(\gamma)} \geq \rho_{\text{max}}. \quad (6)$$

This ensures that the final adapted model strictly respects the sparsity constraint while retaining the most salient task-relevant parameters.

3.4. Stage 2: Salient Structure Adaptation

While Stage 1 effectively identifies the task-essential connectivity pattern, the use of the proximal operator introduces a *shrinkage bias* in the parameter magnitudes. To correct this and achieve precise adaptation, Stage 2 freezes the discovered structure and refines the parameter values within the selected subspace.

First, we crystallize the sparse structure obtained at the termination step t of Stage 1 by extracting a binary mask $\mathcal{M}$ corresponding to the support set of γ^t , effectively locking in the salient parameter directions:

$$\mathcal{M} = \mathbb{I}(|\gamma^t| \geq \tau), \quad (7)$$

where $\mathbb{I}(\cdot)$ is the indicator function and τ is a small threshold (e.g., 10^{-6}) to remove negligible numerical artifacts. This mask rigorously defines the active subspace containing the most critical parameters for the target domain, strictly respecting the sparsity budget $\rho_{\max}$.

Next, we initialize the retrieval parameters $\theta_{\text{ret}}^0 = \theta_{\Delta}^t + \theta_{\text{ret}}$ and perform masked fine-tuning confined to the active subspace defined by $\mathcal{M}$:

$$\theta_{\text{ret}}^{k+1} = \theta_{\text{ret}}^k - \eta \cdot (\nabla_{\theta_{\text{ret}}} \mathcal{L}_{\text{adp}}(\mathcal{D}_t; \theta_{\text{fix}} \cup \theta_{\text{ret}}) \odot \mathcal{M}), \quad (8)$$

where η is the learning rate and $\odot$ denotes the element-wise Hadamard product. By applying the mask to the gradient, we ensure that updates are exclusively applied to the salient connections, preserving the sparsity structure derived from the Inverse Scale Space dynamics.

This two-stage strategy synergizes dynamic structure discovery with precise unregularized optimization, yielding a model that is both structurally sparse and functionally accurate, while remaining parameter-efficient. The whole procedure for both TTDA and OTTA is outlined in Algorithm 1 and 2.

4. Experiments

We evaluate the proposed SDL framework under both test-time domain adaptation and online test-time adaptation settings. Our goal is to assess whether sparse delta learning can identify *more effective adaptation parameters* under a fixed update budget.

Datasets. We conduct experiments on two object recognition benchmarks of varying scales: **Office** (Saenko et al., 2010) (small-scale) and **VisDA-C** (Peng et al., 2017) (large-scale). **Office** is a standard benchmark for domain adaptation, comprising three distinct domains: Amazon (A), DSLR

Table 1. Classification accuracy (%) on the small-scale **Office** dataset under TTDA with ResNet-50. *FC-only* updates all parameters in the bottleneck FC layer (others frozen). Sparse variants (*Random/Magnitude/Ours*) share the same sparsity budget $\rho_{\max} = 0.001$ within the bottleneck FC layer ($\approx 0.0022\%$ of the entire network). Bold text: best performance among sparse variants; Dark background: best performance overall.

Method (Source→Target)	A→D	A→W	D→A	D→W	W→A	W→D	Avg.
Source model only	79.5	72.2	60.8	94.3	62.2	96.6	77.6
Full (Liang et al., 2020)	91.4	90.4	72.2	97.1	73.2	98.6	87.1
FC-only	92.0	91.1	71.6	96.9	71.9	98.8	87.1
Random (Sparse)	77.3	79.0	62.5	97.1	60.2	99.7	79.3
Magnitude (Sparse)	77.1	79.4	62.4	97.1	60.0	100.0	79.3
Ours (SDL)	94.6	92.4	69.3	97.3	69.2	99.0	86.9
Full (Liang et al., 2021)	93.0	93.3	76.0	98.0	75.8	98.2	89.1
FC-only	92.8	92.5	75.3	97.6	75.0	99.2	88.7
Random (Sparse)	92.2	92.1	74.5	95.6	74.5	99.5	88.1
Magnitude (Sparse)	92.2	92.1	74.4	97.5	74.5	99.6	88.4
Ours (SDL)	93.5	93.2	75.5	98.3	75.3	99.4	89.2

(D), and Webcam (W), with 31 classes captured in office environments. **VisDA-C** is a more challenging large-scale benchmark focusing on synthesis-to-real shift. The source domain contains 152k synthetic images rendered from 3D models, while the target domain consists of 55k real-world images from Microsoft COCO across 12 categories.

Backbones and Architecture. Following standard practice (Liang et al., 2020), we use ResNet-50 for Office and Office-Home, and ResNet-101 for VisDA-C. The original classifier is replaced by a 256-dimensional bottleneck layer followed by a task-specific classifier. A Batch Normalization layer is applied after the bottleneck, and the classifier is weight-normalized. During test-time adaptation, the classifier remains frozen.

Compared Methods and Update Strategies. We apply our framework to representative test-time adaptation methods, including SHOT (Liang et al., 2020), SHOT++ (Liang et al., 2021), and IST (Ma, 2024). For a comprehensive comparison, we also include TENT (Wang et al., 2020) and EATA (Niu et al., 2022), which achieve parameter efficiency by updating only the affine parameters in Batch Normalization (BN) layers. To ensure a fair comparison, we adopt the default hyperparameters for all these baselines as reported in their original papers. For each method, we compare four update strategies: (i) the original method with its default dense updates; (ii) *FC-only*, which updates all parameters in the bottleneck layer; and three **budget-matched sparse variants** that update only a fraction $\rho_{\max}$ of the bottleneck parameters, selected by *Random*, *Magnitude*, or *our learned sparse structure*. This design isolates the effect of parameter selection under a fixed sparsity budget.

Evaluation Protocol. For TTDA, we adapt the model using the entire unlabeled target dataset and report classification accuracy on the target test set using the final adapted model. For OTTA, target samples arrive sequentially in mini-batches and the model is updated on-the-fly. We report two metrics: (i) *Post-Update Accuracy (PU-Acc)*, evaluated on each mini-batch after adaptation, measuring immedi-

Table 2. Classification accuracy (%) on the medium-scale **VisDA-C** dataset under *TTDA* with ResNet-101. *FC-only* updates all parameters in the bottleneck FC layer (others frozen). Sparse variants (*Random/Magnitude/Ours*) share the same sparsity budget $\rho_{\max} = 0.0001$ within the bottleneck FC layer (**$\approx 0.0012\%$ of the entire network**). Bold text: best performance among sparse variants; Dark background: best performance overall.

	Method(Synthesis→Real)	plane	bcycl	bus	car	horse	knife	mcycl	person	plant	sktbrd	train	truck	Per-clas
	Source model only	53.7	28.8	50.4	75.0	66.4	4.7	76.3	21.9	68.0	48.6	78.3	6.4	48.2
SHOT	Full (Liang et al., 2020)	94.7	82.7	77.4	58.8	92.3	91.7	84.0	79.0	86.0	88.7	83.9	59.5	81.5
	FC-only	94.6	67.7	80.2	59.3	90.8	66.8	83.9	70.3	80.4	70.2	83.3	40.4	74.0
	Random (Sparse)	90.4	60.0	81.3	52.3	84.1	55.1	82.7	51.7	65.1	58.4	81.7	31.1	66.1
	Magnitude (Sparse)	89.7	59.5	81.1	50.7	82.6	54.2	81.7	51.7	63.2	58.7	81.1	30.6	65.4
	Ours (SDL)	93.9	73.2	74.8	56.2	90.5	11.1	82.2	78.8	80.9	80.0	83.4	51.8	71.4
SHOT++	Full (Liang et al., 2021)	97.8	88.4	89.5	84.2	98.3	98.1	92.4	82.3	96.8	92.0	94.2	34.4	87.9
	FC-only	95.9	81.1	86.1	74.9	95.3	98.4	91.0	83.9	93.6	87.9	85.2	47.2	85.2
	Random (Sparse)	95.5	83.5	84.5	74.7	94.8	96.8	90.5	83.7	92.0	89.4	86.7	48.6	85.2
	Magnitude (Sparse)	95.5	83.4	84.5	74.6	94.8	96.8	90.5	83.7	92.0	89.4	86.6	48.5	85.2
	Ours (SDL)	96.9	81.9	85.9	84.5	95.7	96.6	93.2	83.8	96.1	88.6	89.9	43.8	86.4

ate adaptivity; and (ii) *Final Offline Accuracy (FO-Acc)*, computed by freezing the final model after the stream and evaluating it on the full target test set, reflecting long-term stability and forgetting.

Implementation Details. All sparse variants are applied exclusively to the bottleneck layer, operating under a unified sparsity budget $\rho_{\max} = 0.001$. We employ the SGD optimizer for all methods. For our proposed framework, the threshold τ used to crystallize the binary mask is set to 10^{-6} for *TTDA* and 10^{-4} for *OTTA*. Regarding the baselines, we specify that the *FC-only* variant for TENT and EATA updates the affine parameters of the Batch Normalization layer applied after the bottleneck layer. For sparse selection strategies, *Magnitude* retains parameters with the largest absolute weights, while for *Random* selection, we report the average performance over three independent runs. The batch size is set to 32 for the Office benchmark and 64 for VisDA-C. Comprehensive training details and hyperparameter configurations are provided in the Appendix A.

4.1. Evaluation for Test-Time Domain Adaptation

In this subsection, we evaluate the proposed framework under the standard *TTDA* setting. The classification results on the Office and VisDA-C benchmarks are summarized in Table 1 and Table 2, respectively. Our analysis highlights four key findings:

Superiority over Naive Selection Strategies. First, we demonstrate that effective adaptation under strict sparsity relies on identifying task-essential structures rather than heuristic selection. As shown in Table 1, under a budget of 0.1%, naive strategies like *Random* selection fail to improve upon the source model on challenging tasks (e.g., $A \rightarrow D$), even causing performance degradation (77.3% vs. 79.5%). This gap widens significantly on the large-scale VisDA-C benchmark (Table 2). Under an extreme sparsity of $\rho_{\max} = 0.0001$, both *Random* and *Magnitude* methods

suffer a sharp performance drop to $\sim 66\%$ in the SHOT framework. In contrast, our method maintains a robust accuracy of 71.4%, outperforming naive variants by over 5%. These results validate that our dynamic discovery mechanism effectively filters out irrelevant parameters and locates the critical adaptation subspace, which is unattainable by static or random selection.

Sparsity as Robust Regularization. Second, our results suggest that learned structural sparsity acts as a powerful regularizer, enabling our method to match or even outperform computation-heavy dense updates. This is most evident in the strong SHOT++ baseline. On the Office benchmark, our method achieves an average accuracy of 89.2%, surpassing the fully updated model (89.1%). Similarly, on VisDA-C, while dense methods (*Full* and *FC-only*) plateau at 85.2%, our approach breaks this performance ceiling, reaching 86.4%. We attribute this to the noise-filtering property of the Inverse Scale Space dynamics: by updating only the most salient parameters with persistent gradients, our framework avoids overfitting to noisy pseudo-labels—a common pitfall in dense unsupervised adaptation—thereby bridging the domain gap more effectively.

Gradient Persistence vs. Weight Magnitude. Furthermore, our experiments offer a critical insight into the nature of adaptation: *where to update* is fundamentally different from *which weights are large*. Standard pruning methods, represented by the *Magnitude* baseline, assume that parameters with large absolute values are the most salient. However, as seen in the SHOT results on VisDA-C (Table 2), *Magnitude* selection yields only 65.4%, failing to outperform even the *Random* baseline (66.1%). This indicates that parameters encoding significant source knowledge are not necessarily the ones that require adjustment for domain alignment. In contrast, *SDL* selects parameters based on gradient persistence, accumulating evidence of adaptation demand over time. The superior performance of *SDL* confirms that the

Table 3. Classification accuracy (%) on the small-scale Office dataset under OTTA with ResNet-50. We report PU-Acc (online post-update accuracy) and FO-Acc (final offline accuracy). *FC-only* updates all bottleneck FC parameters (others frozen). Sparse variants (*Random/Magnitude/Ours*) share the same sparsity budget $\rho_{\max} = 0.001$ within the bottleneck FC layer ($\approx 0.0022\%$ of the entire network). Bold text: best performance among sparse variants; Dark background: best performance overall.

PU-Acc (Source→Target)		A→D	A→W	D→A	D→W	W→A	W→D	Avg.
Source model only		79.5	72.2	60.8	94.3	62.2	96.6	77.6
TENT	Full (Wang et al., 2020)	82.8	74.7	64.0	96.7	61.1	99.1	79.7
	FC-only	81.4	73.1	62.0	94.5	62.5	98.5	78.7
EATA	Full (Niu et al., 2022)	78.7	73.8	59.4	95.3	57.3	97.8	77.1
	FC-only	80.8	74.4	62.2	96.6	62.0	98.2	79.0
IST	Full (Ma, 2024)	76.2	59.7	47.9	84.0	44.2	96.2	68.0
	FC-only	74.5	60.8	56.2	84.8	51.0	98.0	70.9
	Random (Sparse)	74.5	60.8	56.2	84.8	51.0	98.0	70.9
	Magnitude (Sparse)	74.5	60.8	56.2	84.8	51.0	98.0	70.9
	Ours (SDL)	80.4	70.4	59.7	92.4	58.3	98.0	76.5
SHOT	Full (Liang et al., 2020)	83.5	76.5	66.0	96.1	63.2	98.9	80.7
	FC-only	83.0	75.1	65.6	95.3	65.3	98.7	80.5
	Random (Sparse)	81.4	73.1	62.0	94.5	62.5	98.5	78.7
	Magnitude (Sparse)	81.4	73.1	62.0	94.5	62.5	98.5	78.7
	Ours (SDL)	83.0	76.0	60.9	95.2	59.4	98.6	78.9
FO-Acc (Source→Target)		A→D	A→W	D→A	D→W	W→A	W→D	Avg.
Source model only		79.5	72.2	60.8	94.3	62.2	96.6	77.6
TENT	Full (Wang et al., 2020)	64.2	47.7	20.7	76.6	19.9	89.8	53.2
	FC-only	70.2	57.9	32.4	83.7	34.4	92.4	61.8
EATA	Full (Niu et al., 2022)	60.3	42.4	17.6	75.1	18.4	88.4	50.4
	FC-only	70.2	57.9	32.5	83.7	34.3	92.4	61.8
IST	Full (Ma, 2024)	72.6	54.9	43.5	77.9	41.3	94.3	64.1
	FC-only	70.8	59.6	54.4	82.6	48.9	97.0	68.9
	Random (Sparse)	70.8	59.6	54.4	82.6	48.9	97.0	68.9
	Magnitude (Sparse)	70.8	59.6	54.4	82.6	48.9	97.0	68.9
	Ours (SDL)	80.7	68.8	57.7	92.2	55.0	97.3	75.3
SHOT	Full (Liang et al., 2020)	84.8	84.5	68.9	96.9	65.7	100.0	83.5
	FC-only	86.3	78.8	67.4	96.0	66.3	99.1	82.3
	Random (Sparse)	83.7	75.5	62.1	95.3	63.2	98.8	79.8
	Magnitude (Sparse)	83.7	75.5	62.1	95.3	63.2	98.8	79.8
	Ours (SDL)	83.9	80.6	59.4	95.8	60.8	98.4	79.8

adaptation subspace is best defined by dynamic optimization signals rather than static weight properties.

Mitigation of Negative Transfer. Unsupervised test-time adaptation carries the inherent risk of negative transfer, where improper updates degrade performance below the pre-trained baseline. This risk is exemplified in the A→D task (Table 1), where naive sparse updates (Random/Magnitude) cause accuracy to drop from the source baseline of 79.5% to $\sim 77\%$. This degradation occurs because unguided updates on irrelevant parameters disrupt the well-calibrated source representations. Our framework effectively acts as a safety mechanism: the Inverse Scale Space dynamics ensure that parameters are activated only when there is strong, consistent evidence from the data. By remaining dormant in the absence of persistent gradients, our method preserves source knowledge where adaptation is unnecessary, thereby preventing catastrophic failure observed in naive strategies.

4.2. Evaluation for Online Test-Time Adaptation

In this subsection, we critically evaluate the proposed SDL framework under the challenging Online Test-Time Adaptation (OTTA) protocol. Unlike standard offline adaptation, OTTA simulates a realistic deployment scenario where the

model encounters target data in a continuous stream and must adapt on-the-fly via a single pass. This strict constraint prohibits access to previous samples, making the model highly susceptible to error accumulation and catastrophic forgetting. To provide a comprehensive assessment, we report PU-Acc (online Post-Update accuracy) to measure immediate plasticity, and FO-Acc (Final Offline accuracy) to assess long-term stability. The results on the Office and VisDA-C benchmarks are presented in Table 3 and Table 4. Our detailed analysis reveals four critical insights regarding the stability-plasticity trade-off.

Topology Matters: Dynamic Sparse vs. Static Dense.

First, we demonstrate that simply reducing the parameter budget is insufficient for stability; the *topological location* of updates is decisive. We compare SDL with TENT/EATA Full and FC-only baselines, all of which update a comparable and negligible fraction of parameters (e.g., $\approx 0.0012\%$ on VisDA-C). As shown in Table 4, while TENT and EATA update BN layers, they suffer catastrophic collapse in the online setting, with FO-Accs dropping to $\sim 15\%$, far below the Source model (48.2%). Similarly, the *FC-only* variants (dense updates restricted to the bottleneck) also fail to maintain stability. This failure indicates that static updates—whether applied to BN or dense FC layers—inevitably disrupt pretrained representations by failing to distinguish between domain-invariant and domain-specific features. In contrast, our SDL identifies a fine-grained sparse topology and maintains a high FO-Acc of 66.3%. This validates our hypothesis: identifying *where* to adapt via ISS dynamics is more critical than *how much* to adapt. Our method acts as a dynamic filter, activating only the “acupuncture points” of the network, thereby achieving effective adaptation without structural disruption.

Safety Valve on “Hard” Tasks (Precision > Scale). Secondly, experimental results reveal that our method exhibits superior stability on “hard” transfer tasks, where existing dense methods fail despite of their higher parameter capacity. We identify tasks like D→A on Office as critical stress tests due to significant distributional discrepancies. As shown in Table 3, TENT Full (updating $\approx 0.2\%$ params) is extremely fragile here, with accuracy collapsing from 60.8% (Source) to 20.7%. This failure stems from the “amplification effect” of dense adaptation: when the domain gap is large, initial pseudo-labels are inherently noisy. Updating a large set of parameters (like all BN layers) based on these noisy signals accelerates error propagation, triggering a domino effect of irreversible negative transfer. In stark contrast, our sparse framework acts as a safety valve. By strictly limiting updates to the discovered sparse structure, we filter out reliable gradients from stochastic noise. Consequently, our method avoids the crash, maintaining a robust accuracy of 59.4% on D→A. This confirms that precision is the key to preventing model collapse in high-risk scenarios.

Table 4. Classification accuracy (%) on the medium-scale VisDA-C dataset under OTTA with ResNet-101. We report PU-Acc (online post-update accuracy) and FO-Acc (final offline accuracy). *FC-only* updates all parameters in the bottleneck FC layer. Sparse variants share the budget $\rho_{\max} = 0.0001$ within the bottleneck FC layer ($\approx 0.0012\%$ of the entire network). Bold text: best performance among sparse variants; Dark background: best performance overall.

Method	PU-Acc (Synthesis→Real)													FO-Acc (Synthesis→Real)												
	plane	beycl	bus	car	horse	knife	meycl	person	plant	sktbrd	train	truck	Per-class	plane	beycl	bus	car	horse	knife	meycl	person	plant	sktbrd	train	truck	Per-class
Source model only	53.7	28.8	50.4	75.0	66.4	4.7	76.3	21.9	68.0	48.6	78.3	6.4	48.2	53.7	28.8	50.4	75.0	66.4	4.7	76.3	21.9	68.0	48.6	78.3	6.4	48.2
IST																										
Full (Ma, 2024)	13.8	60.2	72.4	33.4	25.6	24.2	60.0	54.2	4.9	17.3	43.5	18.7	35.7	10.9	51.2	71.6	29.7	26.1	14.5	57.8	49.0	3.2	15.1	48.5	21.2	33.2
FC-only	73.9	2.6	24.6	33.8	36.8	2.7	80.0	22.8	13.6	9.3	57.9	9.5	30.6	80.4	0.3	1.6	24.0	21.2	0.4	67.6	11.7	1.9	4.1	34.6	3.8	21.0
TENT																										
Full (Wang et al., 2020)	92.5	58.7	85.0	63.5	88.2	78.2	87.7	67.5	78.0	54.9	83.9	24.0	71.8	13.4	8.8	24.7	23.6	20.3	17.3	18.6	16.8	13.0	9.7	20.5	5.4	16.0
FC-only	48.9	35.0	80.9	41.0	49.0	54.5	72.5	20.7	35.4	29.4	71.2	19.0	46.4	0.1	12.0	29.3	19.1	9.6	17.6	23.0	9.9	10.8	7.6	30.6	12.3	15.1
EATA																										
Full (Niu et al., 2022)	87.6	55.1	81.4	50.1	80.3	53.7	80.3	49.7	62.9	57.3	80.8	29.9	64.1	11.6	8.9	21.9	14.7	14.8	14.7	15.3	11.4	10.3	12.8	20.3	10.6	13.9
FC-only	50.4	35.7	79.7	41.0	49.4	54.8	72.5	21.0	36.5	30.9	71.2	19.4	46.9	0.1	12.6	27.7	19.2	10.0	17.6	22.4	10.3	11.1	8.4	30.4	12.9	15.2
SHOT																										
Full (Liang et al., 2020)	93.4	67.3	81.6	60.6	89.5	86.9	83.9	73.1	78.1	68.1	83.0	36.1	75.1	94.5	71.4	82.1	56.1	90.7	90.5	84.4	77.7	80.8	75.7	87.2	42.0	77.8
FC-only	62.1	20.4	81.0	62.3	76.0	87.4	86.1	32.1	54.4	12.2	75.5	5.4	54.6	64.0	21.8	82.5	58.3	78.8	93.7	86.0	41.1	60.3	7.7	78.7	2.9	56.3
Random (Sparse)	50.1	36.8	80.2	41.4	50.1	55.8	72.8	22.0	37.3	29.3	70.8	20.3	47.2	53.5	33.3	77.7	40.1	55.5	61.2	76.4	21.9	41.4	29.2	76.2	20.1	48.9
Magnitude (Sparse)	48.5	36.8	80.2	42.2	50.6	55.7	73.1	21.7	37.0	28.3	71.0	19.7	47.1	51.6	33.4	76.9	41.4	56.5	62.2	77.3	21.2	40.3	28.4	77.3	19.3	48.8
Ours (SDL)	91.0	60.2	78.3	54.4	83.6	78.4	74.2	64.0	74.6	62.6	82.8	27.2	69.3	91.7	47.1	73.9	32.1	86.7	76.5	70.8	72.4	77.9	59.6	89.6	17.8	66.3

From Zero-Sum to Positive Knowledge Accumulation.

A distinguishing feature of our framework is its ability to achieve positive knowledge accumulation in streaming scenarios, breaking the typical “plasticity-stability” trade-off. Most existing online methods (e.g., TENT, EATA, and IST) play a “zero-sum game”: they improve immediate adaptation at the cost of erasing global knowledge. This is evidenced by their *negative adaptation*, where the final model performs worse than the non-adapted Source model (e.g., TENT FO-Acc 53.2% vs. Source 77.6% on Office). In contrast, our method consistently yields an FO-Acc significantly higher than the Source Only baseline (e.g., 66.3% vs. 48.2% on VisDA-C). This result suggests that our Sparse Delta Learning does not merely “survive” the data stream; it actively accumulates global target traits. By locking the majority of parameters and carefully refining a sparse subset, our model behaves like a continual learner, progressively improving performance without suffering from concept drift.

Versatility as a Generic Enhancer. Finally, our framework functions as an objective-agnostic optimization module that seamlessly integrates with, and rescues, diverse TTA paradigms. As shown in Table 3, our method consistently enhances performance independent of the original loss functions. For the struggling IST framework, which inherently produces noisy pseudo-labels in online settings, our sparse mechanism “rescues” the model from collapse, boosting FO-Acc from 64.1% (Full) to 75.3%. Crucially, this benefit extends to the already robust SHOT framework, where our method further refines adaptation stability. This consistent efficacy confirms that our approach serves as a generic enhancer, capable of unlocking the potential of viable adaptation strategies by injecting structural regularization into the optimization process.

4.3. Ablation Study

In this section, we assess the robustness of our framework by examining its sensitivity to optimizer selection and batch

size. Our results indicate that while SGD demonstrates superior stability under strict sparsity constraints in the offline setting, our framework becomes effectively optimizer-agnostic in the online setting, achieving comparable high performance with both SGD and Adam. Regarding the batch size in online adaptation, we find that a moderate size (e.g., 32) yields the optimal trade-off, as it provides sufficient statistics for stable gradient estimation without smoothing out the rapid distributional shifts essential for online adjustment. Detailed numerical tables and comprehensive analyses are provided in Appendix B.

5. Conclusions

This paper introduces Sparse Delta Learning (SDL), a novel framework tailored for the test-time adaptation (TTA) problem. Specifically, we decompose the adaptation process into two coordinated stages: Dynamic Sparse Structure Discovery and Salient Structure Adaptation. In the first stage, our approach effectively leverages Inverse Scale Space (ISS) dynamics to isolate task-relevant parameters while naturally suppressing stochastic noise. Furthermore, an early stopping mechanism is employed to filter out redundant parameters, ensuring strict adherence to a predefined sparsity budget. The second phase marks the transition from structure discovery to functional optimality. In this stage, we construct a binary mask to freeze the identified non-essential parameters enabling targeted optimization solely on discovered salient weights. By integrating the two stages, our proposed SDL framework facilitates precise adaptation within the optimal sparse subspace, thereby alleviating catastrophic forgetting and achieving task-specific functional optimality in TTA scenarios. Extensive experimental evaluations across several benchmarks demonstrate that SDL serves as a plug-and-play framework and can enhance the performance of existing TTA baseline methods, validating its ability to achieve a superior trade-off between adaptation efficiency and predictive accuracy.

Impact Statement

This paper presents work whose goal is to advance the field of Machine Learning. There are many potential societal consequences of our work, none which we feel must be specifically highlighted here.

References

- Agarwal, P., Paudel, D. P., Zaech, J.-N., and Van Gool, L. Unsupervised robust domain adaptation without source data. In *Proceedings of the IEEE/CVF winter conference on applications of computer vision*, pp. 2009–2018, 2022.
- Burger, M., Gilboa, G., Osher, S., and Xu, J. Nonlinear inverse scale space methods. 2006.
- Cao, Z., Li, Z., Guo, X., and Wang, G. Towards cross-environment human activity recognition based on radar without source data. *IEEE Transactions on Vehicular Technology*, 70(11):11843–11854, 2021.
- Chen, D., Wang, D., Darrell, T., and Ebrahimi, S. Contrastive test-time adaptation. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pp. 295–305, 2022.
- Choi, S., Yang, S., Choi, S., and Yun, S. Improving test-time adaptation via shift-agnostic weight regularization and nearest source prototypes. In *ECCV*, 2022.
- Ding, Y., Liang, J., Jiang, B., Zheng, A., and He, R. Maps: A noise-robust progressive learning approach for source-free domain adaptive keypoint detection. *IEEE Transactions on Circuits and Systems for Video Technology*, 34(3):1376–1387, 2023.
- Gandelsman, Y., Sun, Y., Chen, X., and Efros, A. Test-time training with masked autoencoders. *Advances in Neural Information Processing Systems*, 35:29374–29385, 2022.
- Iwasawa, Y. and Matsuo, Y. Test-time classifier adjustment module for model-agnostic domain generalization. *Advances in Neural Information Processing Systems*, 34: 2427–2440, 2021.
- Jang, M., Chung, S.-Y., and Chung, H. W. Test-time adaptation via self-training with nearest neighbor information. *arXiv preprint arXiv:2207.10792*, 2022.
- Liang, J., Hu, D., and Feng, J. Do we really need to access the source data? source hypothesis transfer for unsupervised domain adaptation. In *International conference on machine learning*, pp. 6028–6039. PMLR, 2020.
- Liang, J., Hu, D., Wang, Y., He, R., and Feng, J. Source data-absent unsupervised domain adaptation through hypothesis transfer and labeling transfer. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 44(11): 8602–8617, 2021.
- Liang, X. et al. A comprehensive survey on test-time adaptation under distribution shifts. *arXiv preprint arXiv:2501.xxxxx*, 2025.
- Ma, J. Improved self-training for test-time adaptation. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pp. 23701–23710, 2024.
- Niu, S., Wu, J., Zhang, Y., Chen, Y., Zheng, S., Zhao, P., and Tan, M. Efficient test-time model adaptation without forgetting. In *International conference on machine learning*, pp. 16888–16905. PMLR, 2022.
- Osher, S., Burger, M., Goldfarb, D., Xu, J., and Yin, W. An iterative regularization method for total variation-based image restoration. *Multiscale Modeling & Simulation*, 4(2):460–489, 2005.
- Osher, S., Ruan, F., Xiong, J., Yao, Y., and Yin, W. Sparse recovery via differential inclusions. *Applied and Computational Harmonic Analysis*, 41(2):436–469, 2016.
- Peng, X., Usman, B., Kaushik, N., Hoffman, J., Wang, D., and Saenko, K. Visda: The visual domain adaptation challenge. *arXiv preprint arXiv:1710.06924*, 2017.
- Saenko, K., Kulis, B., Fritz, M., and Darrell, T. Adapting visual category models to new domains. In *European conference on computer vision*, pp. 213–226. Springer, 2010.
- So, A. M.-C., Ye, Y., and Zhang, J. A unified theorem on sdp rank reduction. *Mathematics of Operations Research*, 33(4):910–920, 2008.
- Sun, T., Lu, C., and Ling, H. Domain adaptation with adversarial training on penultimate activations. In *Proceedings of the AAAI conference on artificial intelligence*, volume 37, pp. 9935–9943, 2023.
- Sun, Y., Wang, X., Liu, Z., Miller, J., Efros, A., and Hardt, M. Test-time training with self-supervision for generalization under distribution shifts. In *International conference on machine learning*, pp. 9229–9248. PMLR, 2020.
- Wang, D., Shelhamer, E., Liu, S., Olshausen, B., and Darrell, T. Tent: Fully test-time adaptation by entropy minimization. *arXiv preprint arXiv:2006.10726*, 2020.
- Wang, Q., Fink, O., Van Gool, L., and Dai, D. Continual test-time domain adaptation. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pp. 7201–7211, 2022a.
- Wang, Q., Fink, O., Van Gool, L., and Dai, D. Continual test-time domain adaptation. In *CVPR*, 2022b.

Xia, K., Deng, L., Duch, W., and Wu, D. Privacy-preserving domain adaptation for motor imagery-based brain-computer interfaces. *IEEE Transactions on Biomedical Engineering*, 69(11):3365–3376, 2022.

Zhang, M., Levine, S., and Finn, C. Memo: Test time robustness via adaptation and augmentation. *Advances in neural information processing systems*, 35:38629–38642, 2022.

A. Implementation Details

In this section, we provide comprehensive details regarding the experimental setup, optimization strategies, and hyperparameter configurations for both Test-Time Domain Adaptation (TTDA) and Online Test-Time Adaptation (OTTA) protocols. All experiments are implemented using PyTorch.

A.1. Common Settings

For all experiments, including both baselines and our proposed method Sparse Delta Learning (SDL), we use the **SGD optimizer** with a momentum of 0.9. For the sparse variants, the sparsity budget $\rho_{\max}$ is set according to the benchmark scale:

- **Office:** $\rho_{\max} = 0.001$ of the bottleneck layer parameters ($\approx 0.0022\%$ of the full network; backbone: ResNet-50).
- **VisDA-C:** $\rho_{\max} = 0.001$ of the bottleneck layer parameters ($\approx 0.0012\%$ of the full network; backbone: ResNet-101).

All sparse updates are applied exclusively to the bottleneck fully connected (FC) layer, while the feature extractor remains frozen.

A.2. Test-Time Domain Adaptation (TTDA)

In the offline TTDA setting, the model has access to the full unlabeled target dataset. We compare our method against SHOT and SHOT++ baselines. The specific training configurations are detailed below.

Optimization protocols. For **SHOT**, we follow the standard protocol: the model is trained for 15 epochs, and pseudo-labels are updated once per epoch. For **SHOT++**, which incorporates MixMatch for semi-supervised learning, we follow the original paper: training for 100 epochs on the small-scale Office dataset and for 10 epochs on the large-scale VisDA-C dataset.

Hyperparameters. Table 5 summarizes the learning rates and the specific hyperparameters for our method (α, κ, ν) used in the TTDA experiments.

Table 5. **Hyperparameter configurations for TTDA experiments.** The Stage 1 step size α , damping factor κ , and sparsity coupling ν are specific to our Sparse Delta Learning framework.

Dataset	Method	Learning Rate	Epochs	Ours (SDL)		
				α (Step Size)	κ (Damping)	ν (Sparsity)
Office	SHOT	1×10^{-2}	15	-	-	-
	SHOT (SDL, ours)	1×10^{-2}	15	0.1	1.0	10.0
	SHOT++	1×10^{-2}	100	-	-	-
	SHOT++ (SDL, ours)	1×10^{-2}	100	0.1	1.0	10.0
VisDA-C	SHOT	1×10^{-3}	15	-	-	-
	SHOT (SDL, ours)	1×10^{-3}	15	0.1	1.0	10.0
	SHOT++	1×10^{-3}	10	-	-	-
	SHOT++ (SDL, ours)	1×10^{-3}	10	0.1	1.0	10.0

A.3. Online Test-Time Adaptation (OTTA)

In the online setting, the model adapts to a continuous stream of target data with a batch size of **32** for Office and **64** for VisDA-C. We compare our approach with TENT, EATA, IST, and SHOT (adapted for online use).

Baselines. We adopt the officially recommended learning rates for baseline methods to ensure fair comparison:

- **TENT:** Learning rate 1×10^{-2} (Office) and 1×10^{-3} (VisDA).
- **EATA:** Learning rate 2.5×10^{-4} for all datasets.

- **IST**: Learning rate 1×10^{-3} for all datasets.
- **SHOT (Online)**: Learning rate 1×10^{-2} (Office) and 1×10^{-3} (VisDA).

Our SDL. For our method, we introduce an additional hyperparameter ω (Stage 2 momentum) The specific configurations are detailed in Table 6. Note that the step size α and Stage 2 momentum ω are adjusted based on the stability of the base objective.

Table 6. **Hyperparameter configurations for OTTA experiments.** ω denotes the momentum used in the Stage 2 Salient Structure Adaptation phase.

Dataset	Method Variant	α (Step Size)	κ (Damping)	ν (Sparsity)	ω (Momentum)
Office	SHOT (SDL, ours)	0.5	1.0	1.0	0.5
	IST (SDL, ours)	0.05	1.0	1.0	0.05
VisDA-C	SHOT (SDL, ours)	0.5	1.0	1.0	0.05

*Note: We exclude IST (SDL, ours) on VisDA-C as the IST baseline itself collapses in this setting.

B. Additional Ablation Studies

In this section, we provide detailed numerical results regarding the impact of optimizer selection and batch size on adaptation performance.

B.1. Optimizer Robustness Analysis

Table 7 and Table 8 present the detailed comparison between SGD and Adam optimizers under Test-Time Domain Adaptation (TTDA) and Online Test-Time Adaptation (OTTA) settings, respectively.

Table 7. **TTDA Ablation on VisDA-C**: Classification accuracy (%) using different optimizers and sparsity budgets.

Sparsity ($\rho_{\max}$)	SGD ($1e^{-3}$)	Adam ($1e^{-3}$)
0.001	71.4	63.0
0.003	71.5	68.1
0.005	71.1	70.0
0.007	71.6	70.7

Table 8. **OTTA Optimizer Comparison**: Performance on VisDA-C across different sparsity budgets.

Sparsity ($\rho_{\max}$)	SGD ($1e^{-3}$)		Adam ($1e^{-3}$)	
	PU-Acc	FO-Acc	PU-Acc	FO-Acc
0.001	69.26	66.33	69.16	66.24
0.002	74.65	73.15	74.13	72.91
0.003	75.30	73.73	75.23	74.02
0.005	75.05	74.24	75.85	73.43

B.2. Detailed Analysis on Batch Size

Table 9 demonstrates the sensitivity of our method to batch size variations in the OTTA setting. We observe that a batch size of **32** yields the optimal performance. Interestingly, simply increasing the batch size to 64 or 128 does not result in performance gains; instead, accuracy slightly decreases. This phenomenon suggests that while a certain batch size is necessary for stable gradient estimation, overly large batches may smooth out the local distributional cues required for rapid online adjustment.

Table 9. OTTA Sensitivity to Batch Size: Performance on VisDA-C with sparsity $\rho_{\max} = 0.001$.

Batch Size	PU-Acc (Online)	FO-Acc (Final)
32	71.64	71.25
64	69.30	66.32
128	69.39	69.34